

VantageCloud Lake Architecture

By: Carrie Ballinger

Date: November 6, 2023

Version: A03

Abstract: *VantageCloud Lake offers some key architectural changes to the components that support a Teradata system. This Orange Book explains the key new components and the advantages they provide. It explores the separation of compute and storage, the reduction of planned and unplanned down time, and how familiar database functionalities have been repurposed to provide new value in the cloud. Special emphasis is given to the new Object File System and to Compute Clusters and how they are organized.*

This document is proprietary and confidential to Teradata.

This document may only be used by you in accordance with the terms of a written agreement with Teradata to protect Teradata confidential information. You may not give copies or access to the information in this document to any other person, except pursuant to the terms of a written agreement with Teradata to protect Teradata confidential information. If you do not (or your employer does not) have a written agreement with Teradata to protect Teradata confidential information, discontinue your access to this document now and be aware that any use, disclosure, copying, or distribution of the information in this document is strictly prohibited and is a violation of applicable laws. Please read the detailed copyright notice on the following page.

TERADATA CONFIDENTIAL

Copyright © 2022 by Teradata

All rights reserved.

This document, which includes the information contained herein: (i) is the exclusive property of Teradata Corporation; (ii) constitutes Teradata confidential information; (iii) may not be disclosed by you to third parties; (iv) may only be used by you for the exclusive purpose of facilitating your internal Teradata-authorized use of the Teradata product(s) described in this document to the extent that you have separately acquired a written license from Teradata for such product(s); and (v) is provided to you solely on an "**AS-IS**" basis. In no case will you cause this document or its contents to be disseminated to any third party, reproduced or copied by any means (in whole or in part) without Teradata's prior written consent. Any copy of this document, or portion thereof, must include this notice, and all other restrictive legends appearing in this document. Note that any product, process or technology described in this document may be the subject of other intellectual property rights reserved by Teradata and are not licensed here under. No license rights will be implied. Use, duplication, or disclosure by the United States government is subject to the restrictions set forth in DFARS 252.227-7013(c)(1)(ii) and FAR 52.227-19. Other brand and product names used herein are for identification purposes only and may be trademarks of their respective companies.

Teradata and the Teradata logo are registered trademarks of Teradata Corporation and/or its affiliates in the U.S. or worldwide.

Revision History

Revision/Version*	Authors	Date	Comments
A01	Carrie Ballinger & Rick Glick	07-08-2022	Review version
A02	Carrie Ballinger & Rick Glick	08-15-2022	Final version with review feedback changes
A03	Carrie Ballinger	11-6-2023	Updated terminology and made minor changes to component descriptions

Table of Contents

1	Introduction.....	4
1.1	VantageCloud Lake Architecture.....	4
1.2	Scope.....	4
1.3	Audience	4
2	The Architectural Approach	5
2.1	Primary Cluster	5
2.2	Compute Clusters.....	6
2.2.1	What Compute Clusters Do.....	6
2.2.2	Communicating between Primary Clusters and Compute Clusters	7
2.3	Cloud Object Storage	7
2.4	Shared Services	8
2.4.1	Point of Deployment	8
2.4.2	More on Shared Services	9
2.5	High Availability	10
2.5.1	Unplanned Outages	10
2.5.2	Planned Outages.....	11
3	The Object File System	13
3.1	Benefits of the Object File System	13
3.2	Internal Index Structure	13
3.3	Versioning for an OFS Table	14
3.3.1	Version Numbers.....	15
3.3.2	Use of Internal Index Structure.....	15
3.3.3	Retention Periods	16
3.4	Object File System Table Definition	16
3.4.1	Table Parameters Influencing Access Efficiency.....	16
3.4.2	Assigning Objects to AMPs During Query Processing – No PI Tables.....	16
4	Compute Clusters & Compute Groups	18
4.1	Basics of Compute Clusters	18
4.2	Query Execution on Compute Clusters	18
4.3	Groupings of Compute Clusters	19
4.4	Compute Profiles and Compute Groups.....	21
4.4.1	Min and Max Compute Counts	21
4.4.2	Scaling Policy	22
4.4.3	Compute Profile Start and End Times	22
4.5	Triggering Autoscaling within a Compute Group	23
4.6	Considerations	24
5	Preserving Teradata’s Shared Nothing Advantages	25
5.1.1	The Challenge	25
5.1.2	The “Dynamic” Solution	25
6	Conclusion	27

Table of Figures

Figure 1: A Primary Cluster	5
Figure 3: Many Compute Clusters of Different Sizes.	6
Figure 4: Session Manager, Query Fabric, and Compute Router.	7
Figure 5: Categories of Teradata Object Storage.....	8
Figure 6: Point of Deployment and Multiple Environments.....	9
Figure 7: Shared Services used by a VantageCloud Lake Environment.	10
Figure 8: Blue-Green Deployment.....	11
Figure 9: One Row-Formatted Table's Internal Indexing Structure	14
Figure 10: Modifications made to the Internal Index to Support Versioning.	16
Figure 11: Reading NoPI OFS tables into different sized Clusters.....	17
Figure 12: Query Execution That Spans a Primary and a Compute Cluster.	19
Figure 13: Different sized Clusters in Different Compute Groups.....	19
Figure 14: Compute Group Components.	20
Figure 15: Different Policies for a Compute Group at Different Times of Day.	23
Figure 16: The interplay of Compute Profile Start and End Times Across the Day.	23
Figure 17: Triggering auto-scale within a Compute Group.....	24
Figure 18: Uncoupling the Data Ownership from the AMPs.	25

Table of Tables

Table 1: Traditional Teradata File System vs. New Object File System.....	14
Table 2: Compute Cluster Scaling Policy Implications.	22

Abbreviations and Terms

API	An application programming interface (API) is an interface or communication protocol between different parts of a computer program intended to simplify the implementation and maintenance of software
Autoscaling	Ability to automatically add or remove clusters within a Compute Group
BFS	Block File System, previously referred to as Teradata File System, persistent block storage on the Primary Cluster only
Compute Cluster	A Compute Cluster contains no perm data but can issue I/O against data in object storage; it offloads compute-intensive activities from the Primary Cluster
Compute Group	A defined collection of Compute Clusters owned by a single department or application
Compute Profile	A construct that specifies the number, characteristics and behavior of Computer Clusters for a Compute Group at a specific point in time
Compute Router	The Compute Router directs a request from the Primary Cluster to one of the active Compute Clusters
Environment	A single instance of a VantageCloud Lake configuration or system
Global Throttle	System-managed controls on concurrency that apply to each Compute Group that are automatically set up when a Compute Group is created and can only be defined and changed by Teradata support personnel, not by customers
Hash bucket	One of the outputs of the hashing algorithm, that is used to probe a hash map (an intersection of hash buckets and AMPs) to determine the specific AMP that a row will be inserted into or read from
Internal Object Storage	Object storage owned by the Lake environment, a place where user tables inside the database can be located for cost-effective management
NOS	Native Object Store, means of reading and writing to and from unmanaged object storage
OFS	Object File System, which is similar to the traditional Teradata file system (referred to as BFS), is the new file system that manages tables located in managed storage
POD	Point of Deployment, a collection of specific shared services that support the activity taking place on one or more VantageCloud Lake environments
Primary AMP	Primary AMP, special option for assigning a non-primary index table to an AMP, based on hashing the value of a column from the non-primary index table
Primary Cluster	Primary Cluster contains perm data and shared storage I/O capabilities; a fundamental building block of the VantageCloud Lake architecture
Query Fabric	A communication layer that enables bi-directional work and data movement between one cluster to another within a single VantageCloud Lake environment
Session Manager	A mechanism to route requests from the client application to the Primary Cluster
Snapshot Isolation	Used in managed storage, guarantees that all reads made in a transaction will see a consistent snapshot of the database
External Object Storage	Object storage external to the VantageCloud Lake environment that is accessed using Native Object Store techniques, typically using foreign tables

1 Introduction

This Orange Book provides a simple, high-level view of the VantageCloud Lake architecture.

1.1 VantageCloud Lake Architecture

VantageCloud Lake offers some key architectural changes to the components that support a Vantage system. This Orange Book explains the key new components and the advantages they provide. It explores the separation of compute and storage, the reduction of planned and unplanned down time, and how familiar database functionalities have been repurposed to provide new value in the cloud. Special emphasis is given to the new Object File System (OFS) and to Compute Clusters and how they are organized.

1.2 Scope

This Orange Book provides basic how-it-works information for the foundation of the VantageCloud Lake architecture. It is not intended to be a complete explanation of everything related to the new platform. Out of necessity, this document omits many subject areas related to supporting the VantageCloud Lake platform, such as the automated service activities, pricing, changes to administrative tasks, workload management, and telemetry. A subsequent orange book [Using VantageCloud Lake](#) covers some of these areas as well as greater usability details.

1.3 Audience

This Orange Book is intended for anyone who has a basic technical background and who has an interest in understanding the basics of this new architecture. This includes Teradata technical associates, customer administrators, IT managers, data warehouse architects, data scientists, and application implementers. It assumes some exposure to Teradata technology.

2 The Architectural Approach

The VantageCloud Lake platform was architected with a few key goals in mind:

1. **Preserve and leverage current capabilities of the Teradata database**, such as the mature optimizer, high-performance analytics, scalability, workload management, and high availability.
2. **Repurpose, and extend existing special features** such as MAPS, QueryGrid, table operators, the optimizer, Native Object Store.
3. **Eliminate Planned and Unplanned Down Time** by being resilient and always-on, providing Continuous Delivery and instituting non-disruptive Blue-Green Upgrades for new release implementation.
4. **Leveraging Open Source**, using easy to integrate APIs, secure within the ecosystem, leverages functionality of Cloud Service Providers.
5. **Separation of Compute & Storage**, with heavy reliance on internal and external object storage, providing a cost-effective infrastructure while improving price and performance.
6. **Delivered as a Service**, easy to consume, easy to support, a simplified user experience, easy to purchase and manage budget by individuals, teams/departments, or the enterprise; role of the administrator is minimized and manual steps in provisioning are eliminated.

The following sections highlight the key building blocks of the VantageCloud Lake platform developed to achieve these goals.

2.1 Primary Cluster

The VantageCloud Lake architecture moves Teradata from being a single large database on a single set of BYNET-connected nodes into a logical collection of smaller *clusters* that become the building blocks making up the VantageCloud Lake Platform.

Each cluster is a discrete set of BYNET-connected processing units or nodes. Each node within a cluster contains the familiar AMPs and Parsing Engines just as does any other Teradata database node. Although different clusters do not share the same BYNET, they can communicate by other means (covered later), and a given query may span more than one cluster in its execution.

A *Primary Cluster* is required in all configurations. The figure below illustrates an example of a VantageCloud Lake installation, which uses just a single four-node Primary Cluster. This may be all that is needed in a VantageCloud Lake configuration if the purpose is basic and one-dimensional, such as exploring data that has been placed in object storage.

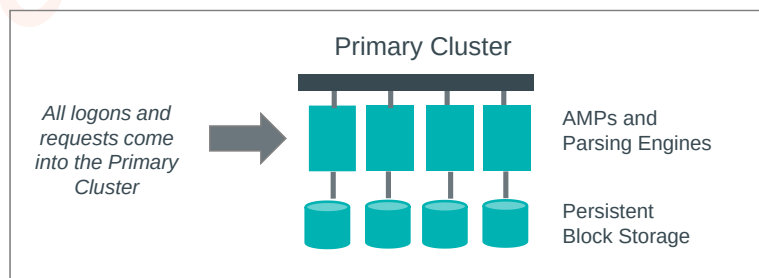


Figure 1: A Primary Cluster

Data can be moved off an enterprise platform and placed on a Primary Cluster as well. A Primary Cluster can be thought of as a small traditional Teradata database with all the same functionality and a very similar architecture. A

Primary Cluster includes a set of permanent nodes whose AMPs own data in persistent block storage managed by the Block File System (BFS). A Primary Cluster also supports the ability to access data in object storage.

The Primary Cluster is responsible for creating the optimized plan for every query executed on the VantageCloud Lake system and determining where each query step should be executed. The Primary Cluster also handles other activities such as session management, dictionary maintenance, privileges, etc. Because it contains persistent block storage managed by the Teradata File System, the Primary Cluster is the preferable location for running short SLA or tactical workloads.

2.2 Compute Clusters

A second type of cluster in VantageCloud Lake is called a *Compute Cluster*. A Compute Cluster is a collection of BYNET-connected compute-only nodes where query work can be sent from the Primary Cluster for execution. Compute Clusters contain AMPs and Parsing Engines, but unlike Primary Clusters, don't have any persistent block storage for user data. Compute Clusters depend on the Primary Cluster optimizer to direct the steps that the Compute Cluster will execute on behalf of a query. No query planning takes place on the Compute Cluster. The Compute Cluster Parsing Engine receives the optimized set of query steps and is responsible for dispatching steps to the Compute Cluster AMPs for execution.

A data dictionary structure exists on a Compute Cluster but has a restricted scope and only reflects the transient needs of that Compute Cluster. It will not initially contain user, table, or column descriptions, nor details about statistics, indexes, or privileges. All structural detail (table definitions, maps, metadata about Compute Clusters) is held in the Primary Cluster.

2.2.1 What Compute Clusters Do

Compute Clusters enable elasticity and extensible compute power in the VantageCloud Lake Platform. Each Compute Cluster stands alone and does query processing for CPU-intensive work such as analytic functions. This frees up the standard Primary Cluster processing units to perform session management, Parsing Engine work, and run SLA or other short work with less contention.

There may be zero or many Compute Clusters in a VantageCloud Lake platform. It depends on the resource needs of the workloads, the volume of requests, and ownership of resources. When to use a Compute Cluster will be determined by the optimizer in the Primary Cluster at the time the initial query plan is built.

During optimization, the work that a query needs to do is broken into several steps. Some of these steps will execute on the Primary Cluster, and if appropriate privileges are in place, some steps will execute on a Compute Cluster. Even if there may be several Compute Clusters within a Compute Group, a single query can currently only execute steps in a single Compute Cluster.

There is a mechanism in place to balance active query steps across Compute Clusters, so no single Compute Cluster is overloaded. Since Compute Clusters are independent and rely on Primary Clusters to send them work, one or more can be added or removed as needed or undergo failure without disturbing ongoing work elsewhere.

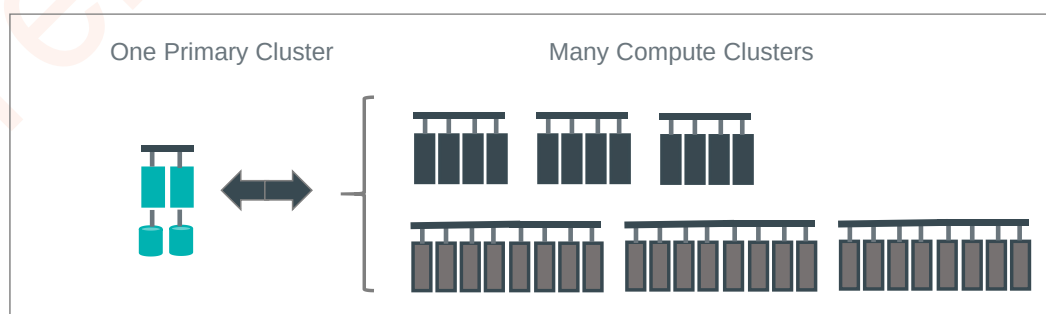


Figure 2: Many Compute Clusters of Different Sizes.

Dedicated collections of Compute Clusters can be provisioned for a department, such as Finance or Marketing. In such a situation, all queries from users that belong to that department will be eligible to execute work on their department's Compute Cluster(s). Automatic addition or removal of processing power, called *autoscaling*, takes place within these departmental groupings of Compute Clusters. A detailed discussion on Compute Clusters and autoscaling can be found in Chapter 4.

2.2.2 Communicating between Primary Clusters and Compute Clusters

Each cluster in a VantageCloud Lake platform is independent and isolated from other clusters. Yet queries can span a Primary and a Compute Cluster, and because of that, will need to communicate with each other.

As illustrated in the figure below, Primary Clusters and Compute Clusters communicate with each other by means of a data access layer referred to as *Query Fabric*. Query Fabric, which does more than move data between clusters, builds upon the existing QueryGrid framework.

In addition to passing data, Query Fabric also passes instructions to the Compute Cluster about how to process the part of the query it will be handling. For example, when Query Fabric moves a spool file¹ from a Primary to a Compute Cluster, information will be passed along with the spool file. This information could tell the Compute Cluster to read data in object storage, then join those results to the spool data from the Primary Cluster.

There are two other components that play a role in communication and enabling sharing among the Primary and Compute clusters.

- **Session Manager** acts as a gateway to all requests coming into the Primary Cluster. It routes the request to the Primary Cluster, selects a node and a Parsing Engine within that cluster, and initiates a session. The Session Manager also plays an important role in ensuring high availability by holding session connections open during replacement of a failed component and to retry queries that return errors.

This component may be the conduit for a large number of concurrent queries at busy customer sites. But even in such cases, the Session Manager is not considered a potential bottleneck. It is very lightweight and has been proven to support over 1000 concurrent queries at the same time with no degradation observed.

- **Compute Router** directs a request from the Primary Cluster to an active Compute Cluster. It is aware of which clusters are active and which are on standby and establishes a session on a specific Parsing Engine within a cluster selected based on routing rules.

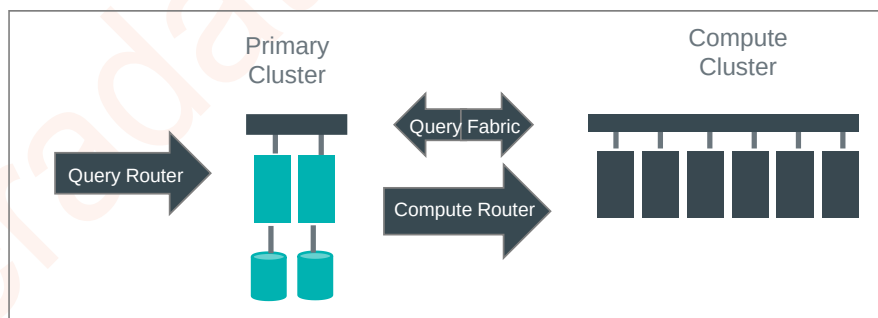


Figure 3: Session Manager, Query Fabric, and Compute Router.

2.3 Cloud Object Storage

Because of the new opportunities in separating compute from storage made available by the cloud, the Primary Cluster itself may end up holding only a small subset of the customer data on its AMPs. The VantageCloud Lake platform lends itself to greater reliance on public cloud vendor object storage for business data. Cloud storage is

¹ A spool file is an internal file that holds temporary data produced in one query step and intended to be read by a subsequent query step.

especially useful for tables that are large, or that support analytics and long-running queries. In addition, it is considered to be more cost-effective.

There are several categories of shared storage accessible by the VantageCloud Lake platform.

- **OFS Storage:** This is the internal type of object storage that is dedicated to user tables that are part of the VantageCloud Lake environment. OFS storage can be viewed and accessed by all clusters within a single environment. Tables in OFS will support inserts, updates, and deletes. OFS is discussed more fully in Chapter 3.
- **External Object Storage:** This object storage is external to the VantageCloud Lake environment, and depending on security conventions, may be shared across multiple other platforms or with trusted third parties. Native Object Store (NOS) access is the method of reading and writing external data that is owned and managed outside the database. Information about Native Object Store and how it works can be found in the orange book titled: *Native Object Store Teradata Vantage Advanced SQL Engine* which can be found at: [Native Object Store Orange Book](#)
- **Private Storage:** Private storage is only accessible by the database code itself. It is the repository for internal database objects such as spool files or database logs. Users cannot access private object storage.

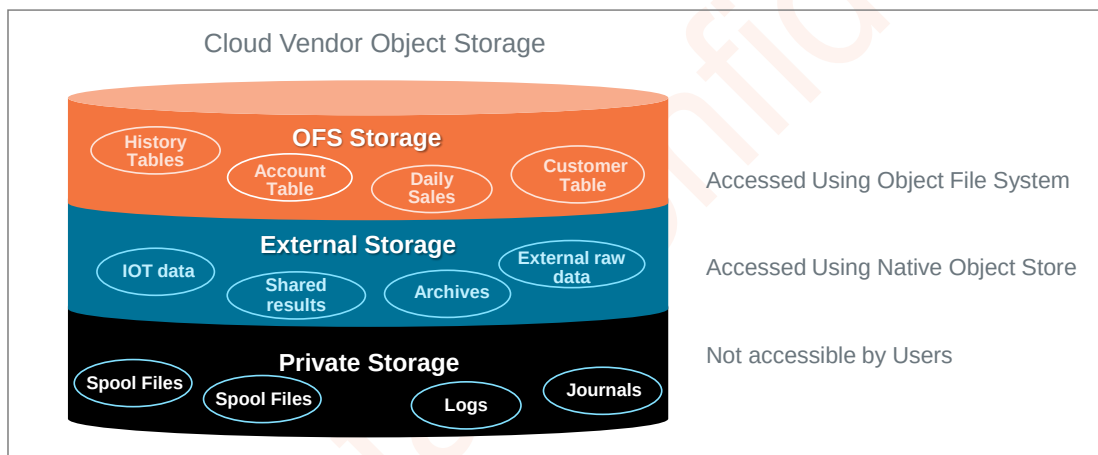


Figure 4: Categories of Teradata Object Storage.

The most important type of object storage from the perspective of the user is OFS storage. This is where existing Teradata tables can be placed to offload them from Primary Cluster BFS storage.

2.4 Shared Services

Discussions so far about the VantageCloud Lake architecture may have left the impression of a multi-cluster configuration that exists as a self-contained system. This is not the case. The database functionality is tightly bound up with a set of pre-defined services that provide essential functionality.

An important requirement for VantageCloud Lake was to be a part of the broader cloud ecosystem and all its wealth of advantages. That includes providing easy access to a variety of tools and software components that do things like managing auto-scaling or provisioning.

2.4.1 Point of Deployment

Each VantageCloud Lake installation is owned by a Point of Deployment (POD) module. A POD is a collection of specific shared services that support the activity taking place on the VantageCloud Lake platform. A POD is owned by Teradata. It is region-specific and provides shared services to multiple single-environment systems. An

environment is the term given to a single system with Primary and (optionally) Compute Clusters. This is also referred to as an *account* in the VantageCloud Lake Console. “Environment” and “account” are terms that both refer to the Primary Cluster-Compute Cluster complex. A single environment shares services with other environments in the same POD. Multiple PODs exist, each independent from each other and each on a private subnet. The environments remain completely secure and separate from each other.

Although multiple environments can be associated with a single POD, each environment is owned by a customer. Environments are protected by separate identity and access management account boundaries and security groups, and act as an SQL endpoint.

Figure 5 below shows the architecture of a POD.

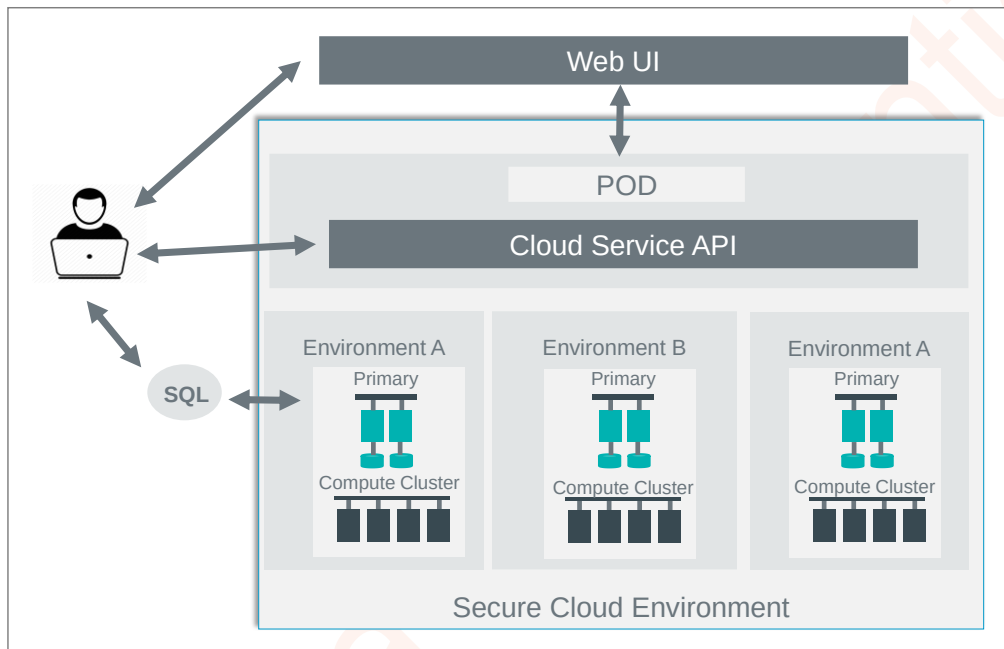


Figure 5: Point of Deployment and Multiple Environments.

2.4.2 More on Shared Services

A set of shared services are shared among multiple environments, in a “multi-environment” relationship. Figure 7 below illustrates a sampling of services that are part of the VantageCloud Lake Shared Services bundle within the POD, as well as a sample of services that are specific to a single environment. If a particular shared service, such as the Upgrade Service, is in high demand the service can be scaled and multiple instances can be active at the same time, each serving a different group of environments.

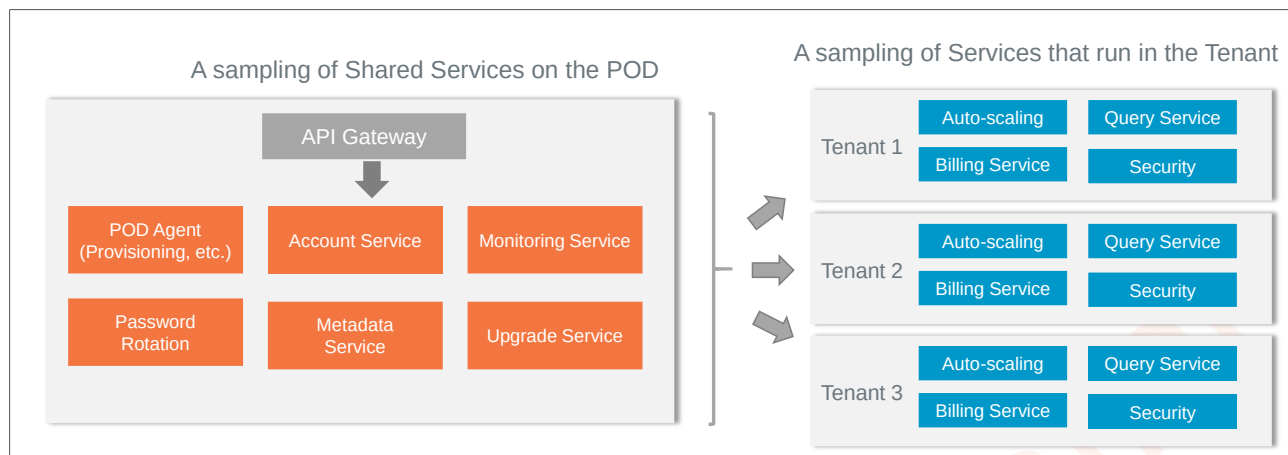


Figure 6: Shared Services used by a VantageCloud Lake Environment.

Many of the services within the Shared Services collection, such as Password Rotation, are only occasionally accessed by a given VantageCloud Lake environment, and therefore do not elicit much demand. Sharing such services across several or many customers is a logistically simple and economical option compared to having a 1:1 pairing between each service and each environment. Any individual service that is experiencing a lot of traffic can be automatically scaled up. When the load goes down on the service, the number of active copies of that service can be automatically reduced.

Infrastructure flexibility is a built-in advantage of this model. While many small environments can comfortably share a single service, some situations may call for dedicated deployments. For example, a large company or a single very large environment could necessitate a dedicated service instance to supply the expected level of traffic.

Another advantage of this sharing model is that services can easily be upgraded or enhanced in a granular way without impacting other services.

2.5 High Availability

VantageCloud Lake offers new approaches to delivering high availability. Database restarts associated with planned and unplanned outages will have minimal impact to clients. Clients will no longer have to re-connect or resubmit queries.

2.5.1 Unplanned Outages

The Session Manager that sits in front of the Primary Cluster handles unplanned outages. The Session Manager is the gateway to all access into the environment and takes the following action in preventing database interruptions when there is an unexpected failure or planned maintenance:

- Sessions active at the time of the restart will be disconnected from the database but will remain connected to the Session Manager.
- Clients will *not* know that they are disconnected from the database but will experience a pause.
- Upon a restart of the database the Session Manager re-establishes sessions with the DBS, all session DDL and DML is re-played.
- In-flight statements are restarted automatically. The user doesn't have to do anything to continue the session.
- An API is triggered for planned maintenance allowing queries to drain.

Note: As more data is placed in object storage, the risk of disk failures on a Primary Cluster's perm node storage is reduced.

2.5.2 Planned Outages

Upgrades and reconfigurations no longer require downtime. There are two upgrade paths that avoid planned outages

1. PAUSE and RESUME
2. Blue-Green Upgrades

When an upgrade is ready to be applied to a VantageCloud Lake system, the PAUSE and RESUME approach is automatically triggered without user initiation. The nodes are upgraded in place with a new image of the software and the user experience is not a disconnect but a slowdown. When the database starts back up, queries are replayed by the Session Manager component, and the user can continue work without interruption.

If there are larger-scale changes, such as dictionary changes or configuration changes, the Blue-Green approach is used. Blue-Green upgrades can be used to provide OS level security updates to all components at one time. They also support changes to the node counts in the Primary Cluster, either expanding or reducing nodes as needed. This eliminates down time and disruptions typically associated with major changes to software or hardware, and is another way that elasticity is improved with the VantageCloud Lake architecture. And finally, the ease of Blue-Green upgrades reduces pressure to choose the “right-sized” system at the start.

An existing system, the Blue system, is replaced by a Green system in order to upgrade the software or hardware configuration of the Blue system. The Green system is deployed and tested as valid while users are still able to access the Blue system. Then upon validation that the Green system is good, users are transparently switched to the Green system and the Blue system is shut down. There is no interruption to user sessions.

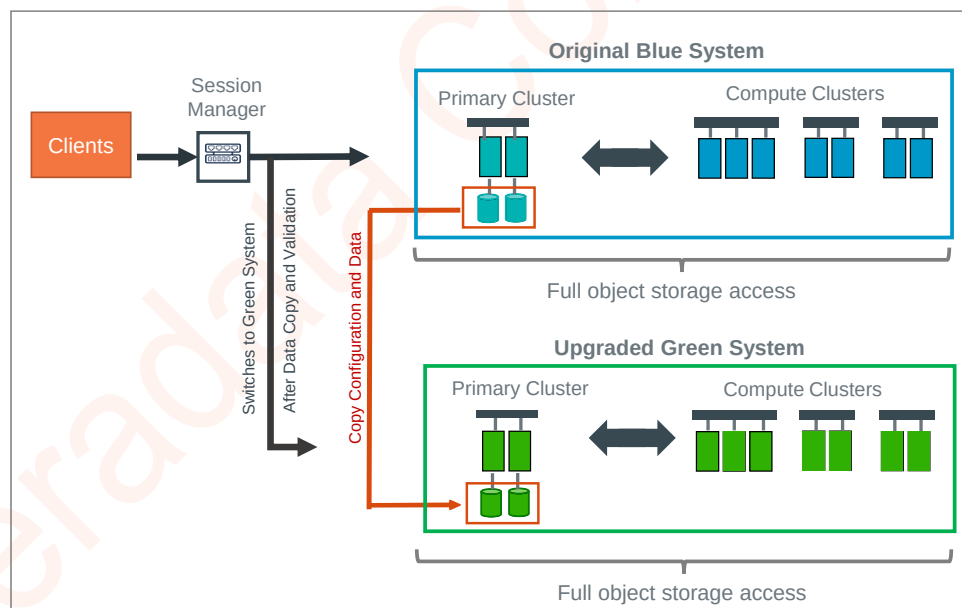


Figure 7: Blue-Green Deployment.

The following events are part of a Blue-Green upgrade:

- A new “Green” configuration is provisioned with latest software or changes to the configuration.
- Data and configuration details from the Blue system's perm storage is written to the Green system while the Blue system continues to process active client requests.

-
- After the perm disk data has been copied, updates that took place on the Blue system while the copy was in process are applied to the Green system.
 - Once the Green system has been validated and logged changes have been applied, the Session Manager switches over to the new Green system.
 - Both systems can access the same data in object storage. The less data in perm storage on the Blue system Primary Cluster, the faster the Blue-Green upgrade will go.

These approaches eliminate the need for planned upgrades that have been associated with earlier Teradata platforms. These high-availability techniques allow customers to be kept up-to-date with the most recent capabilities with little effort or impact.

3 The Object File System

One of the key performance enablers of a traditional Teradata system is its file system, a subsystem that organizes and manages the placement and access of data on the AMP's disks. Called BFS today, this file system is based on each AMP indexing its own data independently from other AMPs, using a master index-cylinder index-data block structure.

To carry over these advantages into the VantageCloud Lake architecture, where much of the data is expected to be in object storage, a new file system has been created called the Object File System (OFS). All data placed in internal object storage, a private and dedicated space for one environment's data, will be under the control of this new file system. Just as in traditional Teradata, all access to table data must pass through the file system's internal indexing structure.

3.1 Benefits of the Object File System

There are several benefits of having OFS:

1. It allows users to issue standard Teradata SQL against data in object storage without syntax changes.
2. Access to data in OFS does not require the definition of a foreign table, as does usual access to data in external object storage.
3. Optimizations have been built in that speed up the access of the data.
4. The tables within OFS are fully ACID compliant.
5. A single writer and many readers will be allowed against the same table.
6. Due to being a versioned file system, time travel becomes available for all OFS tables.

Both row format and columnar format can be used to store and process relational table rows efficiently that are managed by OFS. The default and the preferred approach is columnar/Parquet. Statistics can be collected on OFS tables using similar recommendations as with BFS tables.

3.2 Internal Index Structure

OFS organizes, manages, and indexes user tables similar to how BFS relies on internal index structures. An internal index is automatically built for each table that is created in OFS. This index structure supports quick access and optimal joins. It uses a tree structure with a single root and multiple leaf objects. This construct provides efficiency in multiple ways:

- Offers a means to filter out everything but the objects likely to contain rows of interest
- Pre-positions the rows within each leaf object across AMPs to support parallel access

The data objects contain either data blocks (for row-formatted tables) or column chunks (for columnar tables) that hold the data. See Figure 8.

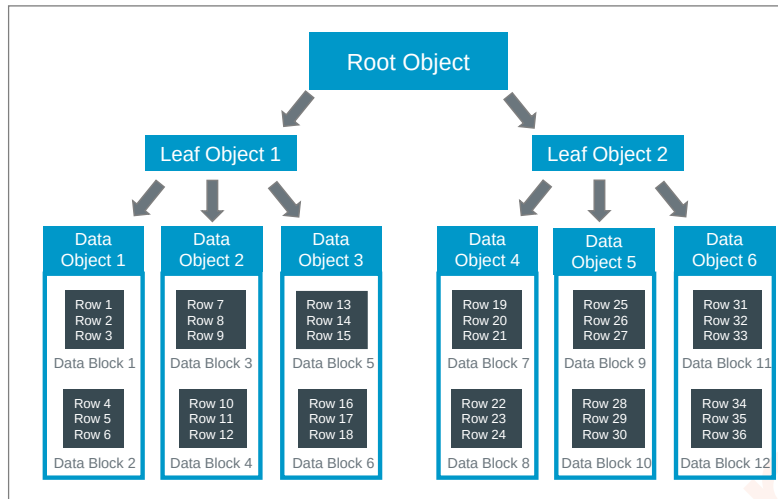


Figure 8: One Row-Formatted Table's Internal Indexing Structure

Similar to how data objects contain rows, the root and leaf objects also hold rows. Each row in the root object carries statistics for each leaf under that root. Each row in a leaf object carries statistics about the data objects that leaf points to.

Min/max values for up to 64 columns per table are carried in each row contained in both the root and leaf objects that constitute that table's index. If the table has more than 64 columns heuristics are used to determine the most relevant columns to include. Because each row in the root represents a single leaf, those min/max value indicate which leaves are pointing to objects whose rows are candidates for satisfying the query.

At the leaf level, the min/max values indicate which objects contain rows that might satisfy query selection criteria. Carrying these min/max values allows filtering to take place within the index structure itself and can greatly reduce the number I/Os required by eliminating objects that will not satisfy the query.

Table 1: Traditional Teradata File System vs. New Object File System.

Traditional Block File System	Object File System
One instance of the file system on each AMP, for all the AMP's data	One instance of the index for each table in managed object storage
Uses master index-cylinder index- data block structure	Uses a root-leaf-data object structure
Located in AMP memory or on the AMP's disk	Stored as separate objects within the database table's bucket

3.3 Versioning for an OFS Table

Versioning is important for two reasons:

1. It supports time travel, which is the ability to access the data in a table as it looked at a previous point in time, using SELECT AS OF syntax.
2. Allows transactionally consistent reading to occur while write operations are taking place.

3.3.1 Version Numbers

Tables added to the Object File System are managed by an internal method of versioning. A version number is assigned to the table itself, but there is no versioning applied to individual rows within the table, or the objects that hold those rows.

When an OFS table is first created it is given a Version of 0, but after the initial load of data into the table the version is changed to 1. Any subsequent change to the table, however small it might be, will cause a new version of that table to be created. This does not mean an entire duplication of the table is made, however. Only the new pieces are appended.

All objects have an assigned name. Objects that contain data rows, leaf objects, and root objects all come with names. The current table version is carried in the Data Dictionary in the DBC.TVM table, along with the name of the root object associated with that version number. There may be many root objects for a given table, each representing a different version of the data.

3.3.2 Use of Internal Index Structure

When performing time travel, the root associated with the time of interest is selected among the collection of root objects that exist for that table. That root is connected to leaf objects that point to the data objects as they looked at that particular point in time. Time travel is enabled by using the SELECT AS OF syntax in a query. When such a query is submitted the root object that matches the point in time specified in the query will determine the data that the user will see when the answer set is built up.

A table in the OFS can be updated after the initial load, however existing objects are never modified. This could be a batch update using INSERT SELECT, or it could constitute row-at-a-time INSERT, DELETE, or UPDATE commands. In all cases, new objects are written to reflect the changes and no objects are updated in place. For example, the addition of a single new row results in the creation of a new object, just for that single row. Once that new object is created, a new leaf object is created above it, in order to reference the new data object. In addition, a new root object is created to incorporate that new leaf object. The DBC.TVM table is then updated to reflect a +1 version, and the name of the newly created root object is added to the TVM row associated with the new version of the table.

Figure 9 shows how the root-leaf index structure supports versioning when a single row is updated. An update to a single row requires the deletion of the old row and the insertion of a new row. This will result in two objects having to be created 1.) for the rows in the object that were not changed, and 2.) for the newly updated row.

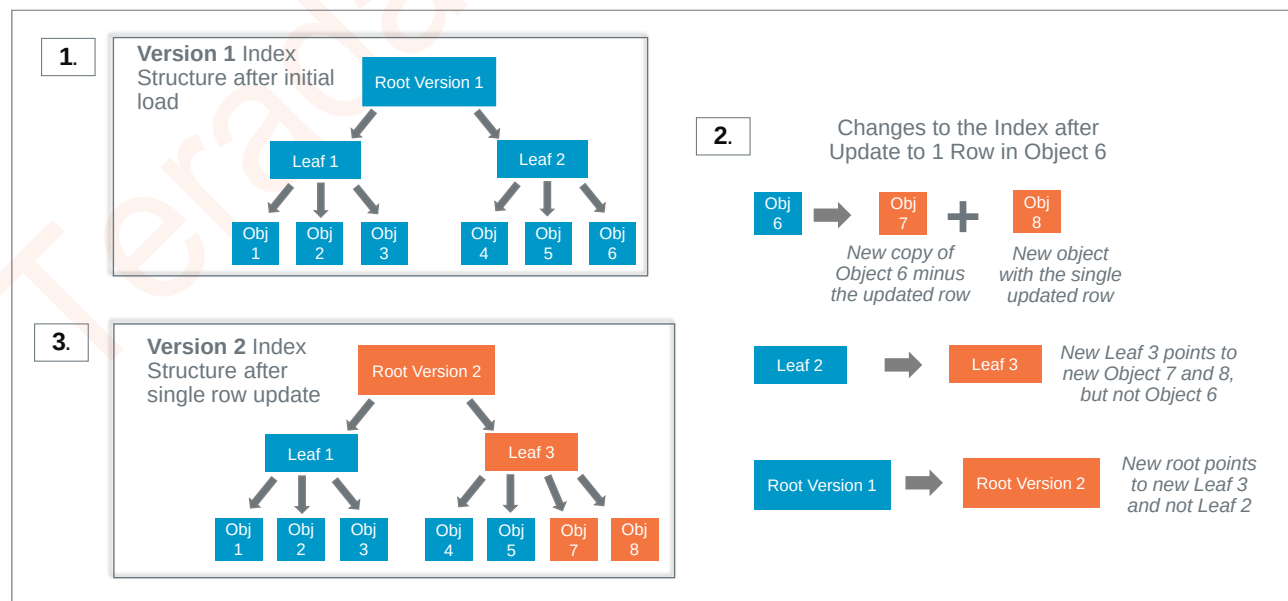


Figure 9: Modifications made to the Internal Index to Support Versioning.

The root object is the primary driving point for versioning. Underneath a newly created root object will be the new leaf object(s) and the new data objects, as well as all the unchanged leaf objects and unchanged data objects that were *not* impacted by the recent update. Each root sits at the pinnacle of a point in time collection of already-existing leaf objects and data objects, as well as the new leaf object(s) and their new data objects.

3.3.3 Retention Periods

Objects that make up a version of an OFS table are not kept indefinitely. A purge process removes outdated or obsolete versions of tables, subject to a retention policy. By default, the retention period 30 days, after which the version of the table is automatically dropped.

A key benefit of this versioning approach is that the writer task is operating on new objects, and for that reason will not block any reading tasks. A table-level write lock is placed when updating an OFS table to prevent concurrent writes. But readers of an OFS table place an access lock instead of a read lock. Readers will access the last committed version of the data, based on what is current in the DBC.TVM table.

It is recommended that rows in OFS be updated using an INSERT SELECT as much as possible so new objects that reflect the changes will contain more rows and be more fully sized.

3.4 Object File System Table Definition

There are different options that can be considered when creating a table within BFS, for example, block-level compression. Similarly, there are several considerations when creating a table in OFS, but they are somewhat different. This section covers a couple of the different parameters involved in creating an OFS table.

3.4.1 Table Parameters Influencing Access Efficiency

In a similar way to how rows are assigned to AMPs in BFS, the table definition itself will determine how the OFS table rows will be stored across objects. Below is a brief description of some of the parameters on the CREATE TABLE statement that can be used when the table will be placed in managed object storage:

ORDER BY: An ORDER BY clause specifies on what columns the data is sorted for an INSERT-SELECT into an OFS table. The source spool of an INSERT-SELECT is sorted and is distributed based on the value range of the ORDER BY columns. This value-distributed spool is then used by the AMPs to build data objects. Each AMP processes its rows from the spool to construct new data objects at 16MB boundaries.

PRIMARY INDEX (PI): The column(s) specified in the PRIMARY INDEX clause will act similar to a PRIMARY AMP² specification for a columnar table. The capability is not yet in VantageCloud Lake.

The ORDER BY option is available currently, but the PRIMARY INDEX parameter is not available at the time of this orange book's publication.

3.4.2 Assigning Objects to AMPs During Query Processing – No PI Tables

In traditional Teradata, each AMP owns its share of the data. Which AMP owns which rows is determined by hashing the primary index (or first column in the table) of each row and using the hash bucket that is produced to probe a hash map table. A hash map intersects hash buckets and AMPs in as even a way as possible. BFS attempts to evenly spread the rows of a table across all the AMPs in the map where the table is defined. The degree of even distribution of rows across AMPs also will be influenced by the choice of the table's primary index.

When reading data in object storage, AMPs are not directly associated to the data. This is true whether the AMPs doing the reading are in the Primary Cluster or the Compute Cluster. It is also likely that different clusters with

² Primary AMP is special option for assigning rows of a non-primary index table to an AMP, based on hashing the Primary AMP column value.

different numbers of AMP may be reading the table at different times. Consequently, a mechanism is needed to assign the objects to be read to the AMPs that are doing the reading. And this object-to-AMP assignment mechanism will have to be flexible.

Consider a common case where an OFS table does not have a primary index. That table will be considered a No Primary Index (NoPI) table. In such a case AMPs are assigned to read a portion of the total objects by relying on Teradata's hashing algorithm. The unique object identifier of each selected object is put through the Teradata hashing algorithm and a hash bucket is produced. That hash bucket is used to probe a dynamic hash map that has been created on-the-fly based on the number of AMPs of the cluster doing the reading.

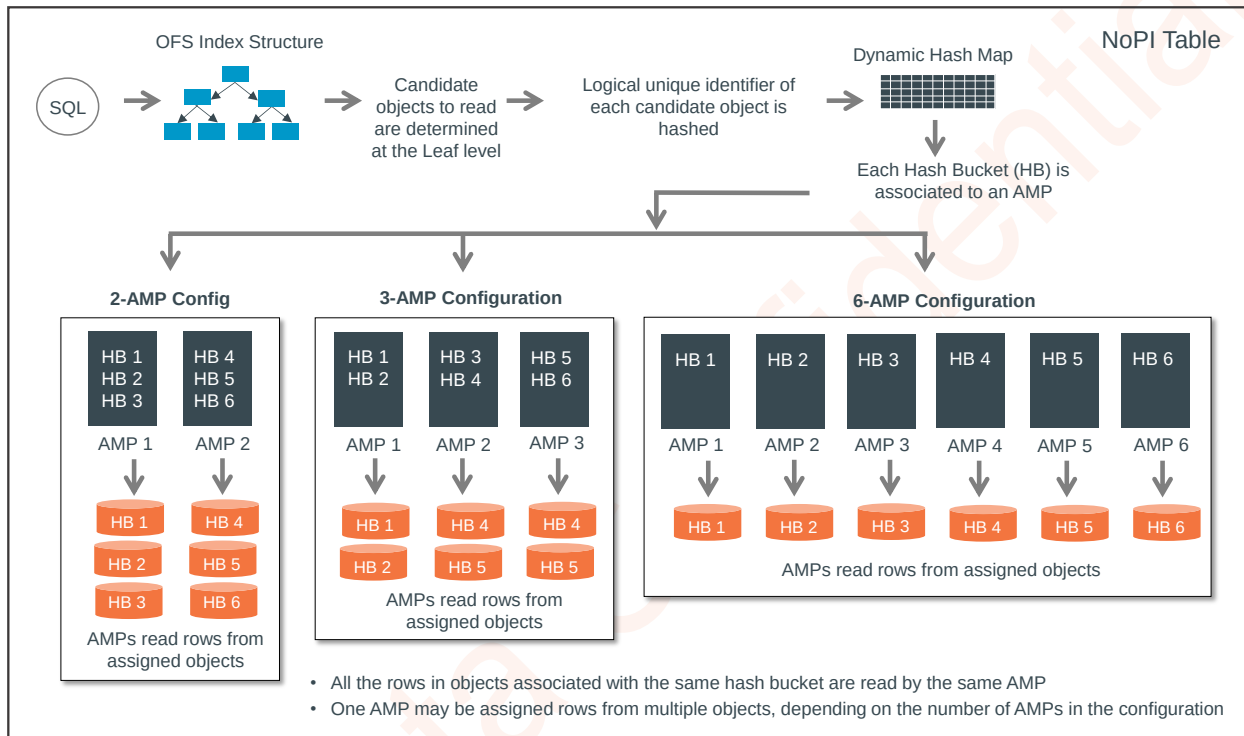


Figure 10: Reading NoPI OFS tables into different sized Clusters.

4 Compute Clusters & Compute Groups

It is anticipated that most of the work running on the VantageCloud Lake platform will be executing on the Compute Clusters. This chapter explains what Compute Clusters and Compute Groups are, some background conventions, and considerations for making the best use of them.

4.1 Basics of Compute Clusters

A Compute Cluster is a set of BYNET-connected Teradata nodes that are intended to perform compute-intensive work. They contain AMPS and Parsing Engines and a data dictionary, but do not contain persistent storage for user data.

There are several benefits of Compute Clusters in the VantageCloud Lake architecture:

1. **Autoscaling:** The Compute Cluster is the unit of elasticity; they can be scaled up and down within the Compute Group.
2. **Protection of Primary Cluster Work:** Offloading compute-intensive processing and in-database analytics to the Compute Clusters reduces contention for the SLA work running on the Primary Cluster.
3. **Departmental control:** Groups of Compute Clusters can be assigned to individual departments or applications, giving control over cost, resource usage, and performance to a local team, rather than a centralized administrator.
4. **Isolation of work:** Each Compute Cluster is working independently and work running there will not compete for resources with other work that is running in a different cluster. Exploratory efforts against unfamiliar data can be given its own set of Compute Clusters apart from more disciplined reporting applications. A special grouping of Compute Clusters could be set up for Data Labs requests. Another set of Compute Clusters could be set up for requests that use a particular BI tool, such as Tableau.
5. **Resiliency:** If a Compute Cluster fails, this has limited impact, the entire system is not brought down. Only the cluster involved undergoes replacement and in-flight queries running on that cluster are automatically resubmitted. The user only experiences a modest pause.

Compute Clusters come in two types: Standard clusters and Analytic clusters. The Analytic Cluster is a specialized compute cluster for executing user scripts within containers against table data using the Open Analytic Framework. Analytic Clusters may also be used for other purposes, for example users may execute regular SQL queries on these clusters.

What makes Analytic Compute Clusters different is that each query executing on them is given more memory. The query limit on analytic clusters is lower than on the standard clusters and there is more CPU per AMP available for each query's execution. For more information on Analytic Clusters and how to use them, see Section 6.4 in [Using VantageCloud Lake Orange Book](#).

4.2 Query Execution on Compute Clusters

It is important to emphasize that all queries entering a VantageCloud Lake platform are first received by the Session Manager and assigned to a Parsing Engine of a Primary Cluster. The optimizer on that Primary Cluster will look at where the tables being accessed are located and build the query plan accordingly. This plan may involve executing a step or two in the Primary Cluster itself, and then passing control to a Compute Cluster. All final result sets generated on a Compute Cluster are returned to the Primary Cluster to be sent back across the Session Manager to the client.

For the initial releases of the VantageCloud Lake platform, query steps that need to be executed on the Primary Cluster will be grouped together and executed in the beginning of the query. Then control will be passed to the

Compute Cluster, and all the query steps to be done on the Compute Cluster will be performed there in a single batch. There will be no overlap between the execution of the two clusters' query steps. Control always returns to the Primary Cluster for the final sort and returning the answer set to the client.

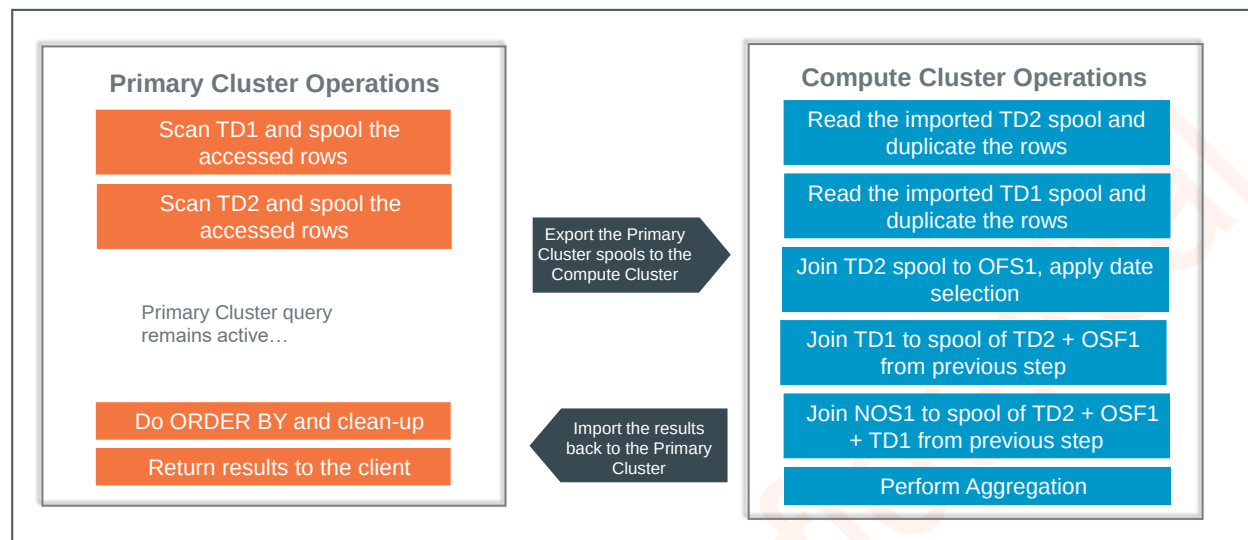


Figure 11: Query Execution That Spans a Primary and a Compute Cluster.

Steps expected to need a lot of memory, such as steps that execute R or SCRIPT table operators, are eligible to be executed on specific types of Compute Clusters with high-memory nodes, referred to as *Analytic Clusters*.

4.3 Groupings of Compute Clusters

When multiple Compute Clusters are dedicated to a single department or application, this is known as a *Compute Group*. Compute Groups are the foundation of a simple and cost-effective scale-up architecture that can be customized for each Compute Group in a distinct way.

Different departments may have their own Compute Groups, with different sizes and numbers of clusters, as illustrated in Figure 12.

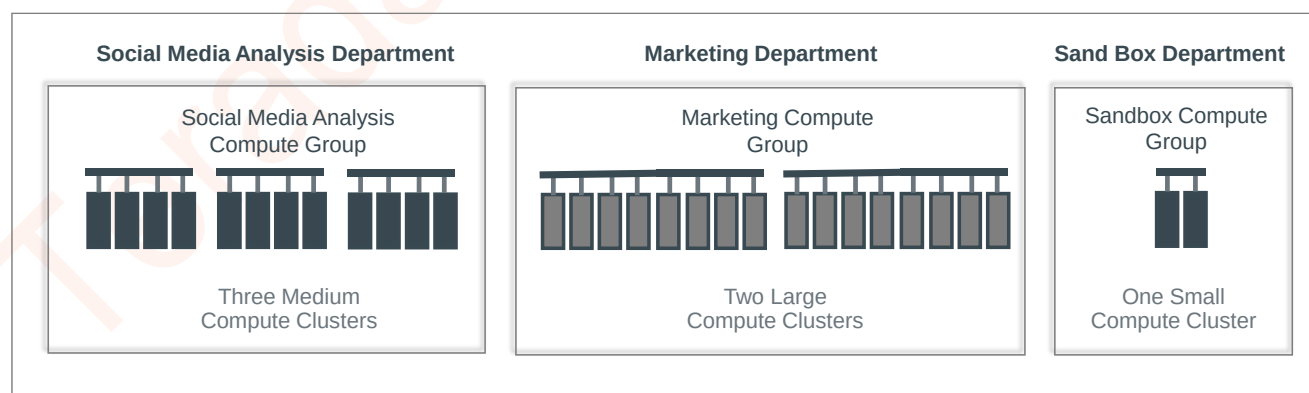


Figure 12: Different sized Clusters in Different Compute Groups.

The characteristics of the clusters within a Compute Group at any point in time is specified in a construct known as a *Compute Profile*.³ The Compute Profile defines the number of Compute Clusters and the upper limit on auto-scale for the Compute Group. It specifies the size of each cluster and can be given a time window (start and end time). There may be multiple Compute Profiles owned by a Compute Group, each with different specifications.

Figure 13 illustrates the relationship between Compute Groups, Compute Profiles, and Compute Clusters.

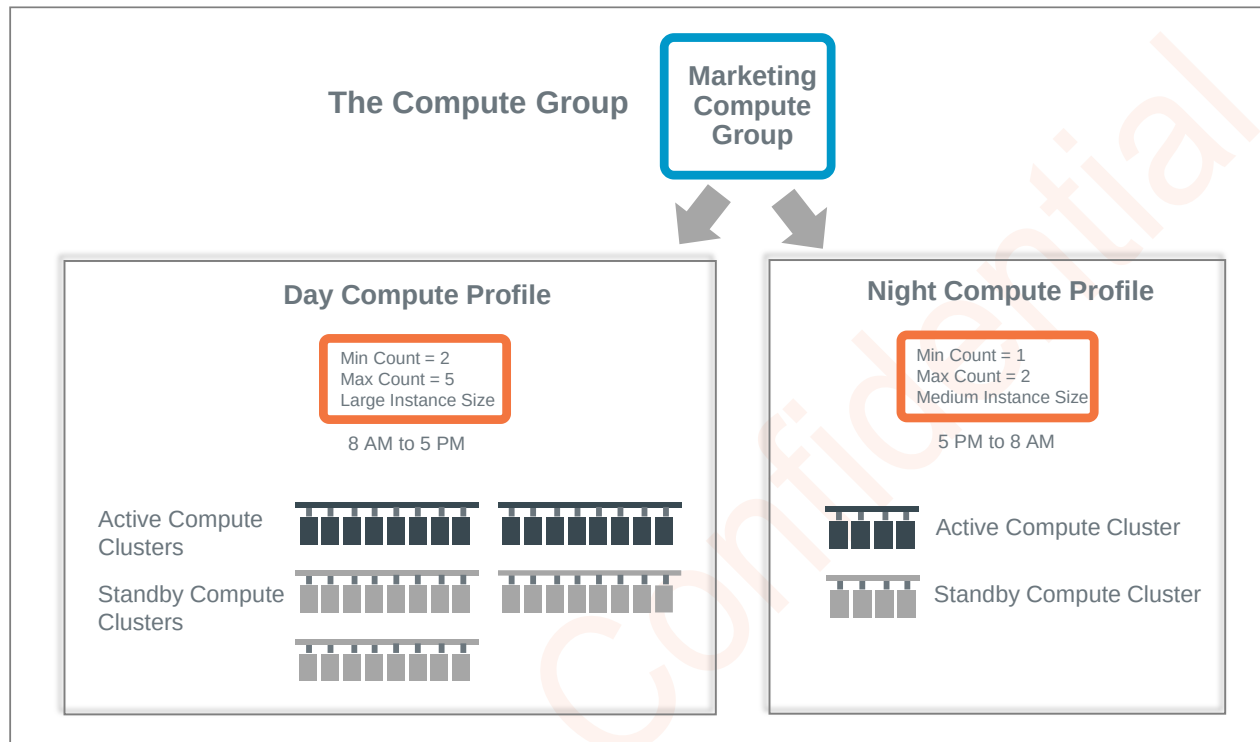


Figure 13: Compute Group Components.

Queries are routed to Compute Clusters based on information in the user's User record or Profile record, or by means of a session parameter.

There could be an advantage of having more than one Compute Groups within a single department, with each Compute Group supporting a different function or type of work. Compute Groups could be set up for the Marketing department just for users who run advanced analytics or table operators frequently, and another Compute Group for users from Marketing that are running standard reporting.

T-shirt sizes are used to represent the size (number of nodes) of each of the Compute Clusters within a given Compute Profile. T-shirt size is referred to as "Instance Size" or just "Instance" in the SQL and in the Vantage Console screens used for provisioning Compute Groups and their Compute Profiles. In the Day Compute Profile above each Compute Cluster has a Large T-shirt size (8 nodes), while in the Night Compute Profile each Compute Cluster has a Medium T-shirt size (4 nodes).

³ Be aware that a User Profile is a different entity from a Compute Profile. The former defines a set of parameters (such as spool space or an account string) that can be assigned to a group of users; the latter is specific to the VantageCloud Lake database and carries definitional information about Compute Groups during a specific window of time.

4.4 Compute Profiles and Compute Groups

The Compute Group is the parent or owner of the Compute Clusters to be available to the department or an application. However, most of the parameters that influence Compute Cluster characteristics and behavior are defined in a Compute Profile.

Key parameters defined by the Compute Profile (as they are labeled in the DDL) include:

- **Instance Size:** Describes the number of nodes within each Compute Cluster in the Compute Group using T-shirt sizes. A sampling of the smaller T-shirt sizes are shown below:
 - Small – 2 nodes
 - Medium – 4 nodes
 - Large – 8 nodes
- **Min Compute Count and Max Compute Count:** Min Compute Count defines number of clusters that are always active in the Compute Group when this Compute Profile is active. The Max Compute Count represents the upper limit for autoscaling, the maximum number of clusters the Compute Group is allowed to have active. In this document the term “standby” is used to describe Compute Clusters that are between the min and max values. Standby clusters are inactive initially but are available to be activated if demand within the Compute Group should increase.

Note: In the Vantage Console provisioning screens, “Min Compute Count” and “Max Compute Count” are expressed as “Min Instance Count” and “Max Instance Count”.
- **Scaling Policy:** Directs specific details of the autoscaling procedure for active Compute Clusters. See Table 2 below for the current options.
- **Start and End Time:** An optional setting that allows the administrator to specify a start time and end time for the Compute Policy, useful when different windows of time require different setup options. Cron expressions can be used to indicate when the Compute Cluster policy starts and ends, if SQL commands are used to define Compute Profiles.
- **Cool-Down Time:** The period of time a Compute Cluster will continue to remain active beyond the Compute Profile’s specified End Time so in-flight queries can complete. No new queries will be accepted by the clusters defined by the Compute Profile during the cool-down period.

A single Compute Profile that offers autoscaling may satisfy operations around the clock for many departments, especially when getting started. The next section provides more details for each of the parameters with the Compute Profile.

Because nodes contain AMPs, the instance size (number of nodes in the Compute Cluster) defined in the Compute Profile controls the degree of parallelism possible within each Compute Cluster. To achieve greater parallelism, very large VantageCloud Lake installations may select one of the larger sizes which will provide greater number of nodes within each Compute Cluster.

4.4.1 Min and Max Compute Counts

Standby Compute Clusters are initially inactive but are available for autoscaling as needed. Specification of active and standby Compute Clusters is done by means of the Min and Max Compute Count parameters within the Compute Profile. The difference between the Min and the Max values establishes the degree of elasticity within the Compute Group.

If there is a desire to eliminate all autoscaling capabilities for a particular department, set the department’s Compute Profile so that the min and the max count values are equal. This may be appropriate for Compute Groups supporting Data Labs or other less critical or activities.

4.4.2 Scaling Policy

One of the parameters in the Compute Profile is called “Scaling Policy”. The default scaling policy (Standard) is expected to be appropriate for general use. Currently, Standard is the only scaling policy option available, but in the future there may be other scaling policy choices.

On AWS configurations, all Compute Clusters are provisioned at the time of the initial Compute Profile creation. The standby clusters (clusters supporting autoscaling as specified in the Compute Profile), are put into a hibernation state immediately. When autoscaling requires an additional Compute Cluster, the time to bring a new standby Compute Cluster into an active state is reduced.

This table describes the two scaling policy options (which are listed across the top row) and how they behave during scale out and scale in for AWS.

Table 2: Compute Cluster Scaling Policy Implications.

	Standard Scaling with AWS (Default)
During scale-out	Standby Compute Clusters have been kept in a hibernated state to reduce provisioning time during scale-out
During scale-in	Standby Compute Clusters will be drained of active queries before being returned to a hibernated state

On Azure configurations, the standby clusters are not provisioned upfront. Instead, the clusters are creating during scale-out time. During scale-in, the cluster is terminated instead of hibernated.

4.4.3 Compute Profile Start and End Times

The purpose of Compute Profile Start Time and Compute Cluster End Time parameters is to allow different policy parameters to be in effect for the same Compute Group during different windows of time. For example, more processing power may be needed for daytime hours, while much less processing power is needed during night hours.

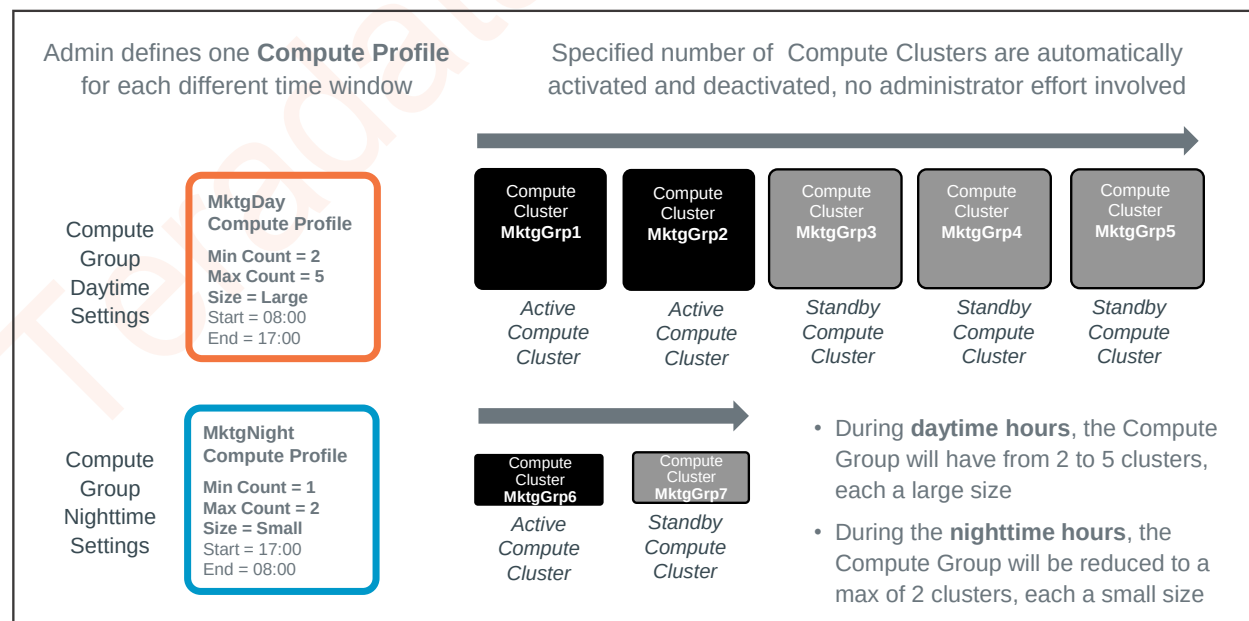


Figure 14: Different Policies for a Compute Group at Different Times of Day.

Figure 14 illustrates how two Compute Profiles for two different time windows might look. Compute Profile start and end time flexibility is targeted to cost-conscious customers who need different Compute Cluster settings during a specific range of hours.

Although not shown in the figure above, another parameter that can change by time of day is the scaling policy. For example, it may be more important during the daytime hours to keep standby Compute Clusters that are not in use in a suspended state to reduce provisioning time when they are spun up.

When setting up Compute Profiles, consider slightly overlapping the time windows to allow time to activate new Compute Clusters and time to drain the work on previously active Compute Clusters. Figure 15 illustrates how a Large Compute Profile and a Small Compute Profile hand off control across a 24-hour period.

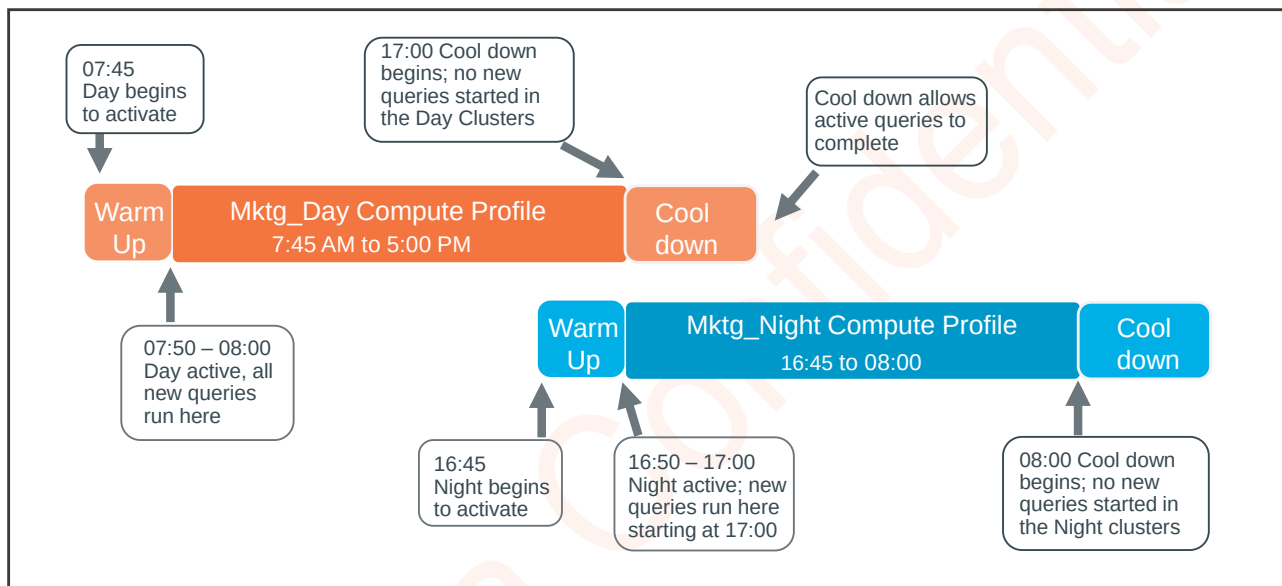


Figure 15: The interplay of Compute Profile Start and End Times Across the Day.

Note: If different Compute Profile start and end times overlap, that is, more than one Compute Profile is active at the same time, the Compute Router will always send work to the largest clusters. In Figure 15, even if the night profile clusters are up and running by 16:50, all query work will continue to be sent to the day profile until 17:00, when the day profile's cool down period begins.

The above example shows how automated time-based changes in Compute Profiles work. Customers also have the option to create their own more flexible schedules based on APIs. For example, customers who have a nightly batch process can bring up a Compute Cluster when they are ready to initiate the batch process and tear it down immediately after the process completes.

4.5 Triggering Autoscaling within a Compute Group

Once one or more Compute Profiles have been defined, elasticity within a Compute Group takes place automatically. Scaling out or scaling in does not require administrator involvement or any additional definitional effort.

There are several methods of triggering the addition or subtraction of a cluster within a Compute Group.

1. **Global Throttles:** Global Throttles concurrency control mechanism that are automatically set up when a Compute Group is created. Global Throttles can only be defined and changed by Teradata support personnel. By default, the Compute Group Global Throttle will restrict the number of queries allowed to start executing to 60 for each standard cluster in the Compute Group. If a query enters the system that would cause

that throttle limit to be exceeded, that query will be placed in a delay queue until one of the active queries within the Compute Group completes.

The act of placing a query in the delay queue sets in motion an auto-scale action within the Compute Group assuming the Max Compute Count as defined in the Compute Profile has not been reached. If there are standby Compute Clusters available, one of them will be activated and begin accepting work. If there are no additional standby clusters available, then the query will stay in the delay queue until such time as one of the active queries within a Compute Group's clusters completes.

2. **CPU Utilization:** If the average CPU usage of active clusters within the Compute Group reaches a high percentage of CPU, for example 95%, this will also trigger an auto-scale action. Because it is possible to reach that threshold of CPU independent of the Compute Group's Global Throttle limit, it is possible for concurrency to be below the throttle threshold, and yet have an additional Compute Cluster activated as shown in Figure 16.
3. **External Orchestration:** The administrator, or an external tool, can trigger Compute Cluster deployment or hibernation at specific times using APIs, if greater management of capacity within the Compute Group is required.

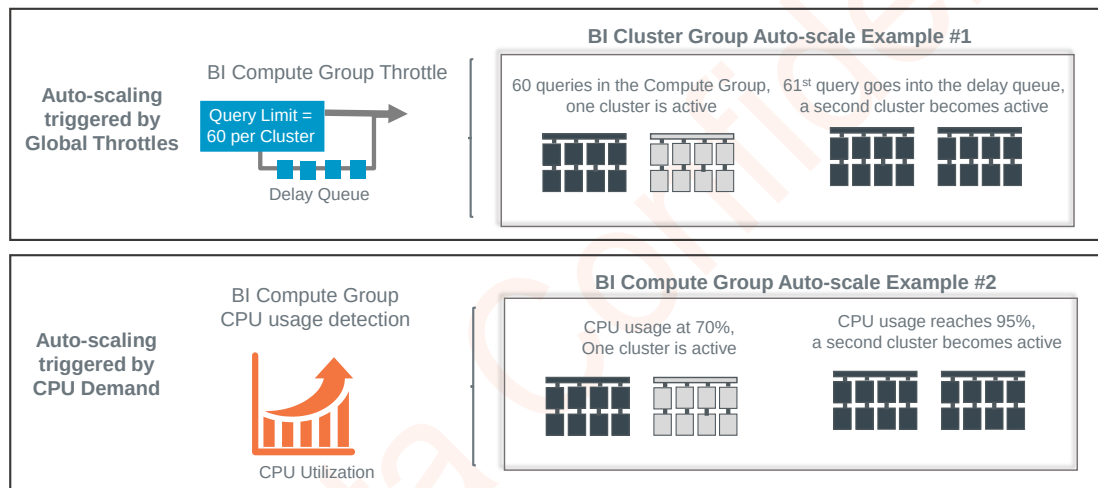


Figure 16: Triggering auto-scale within a Compute Group.

4.6 Considerations

Below are some further explanations concerning Compute Clusters and Compute Group setup:

- Parameters specified in the Compute Profile eliminate the need to create each individual Compute Cluster separately. In addition, a new set of Compute Clusters will be brought up automatically when a Compute Profile becomes active. Compute Clusters supporting a Compute Profile that has just ended will be discarded.
- The Max Compute Count parameter is an important mechanism for keeping autoscaling at a reasonable level and proactively managing cost for the department. If the Min and Max Compute Counts are set to be equal, no autoscaling will take place.
- Even though Compute Profile parameters has been defined for a specific time span, this can be overridden by the DBA using SQL statements. Using SQL, Compute Profiles can be suspended and resumed at any time. However, consider using this override option only in exceptional situations, because unexpected cost overruns or surprise performance degradation could result.

5 Preserving Teradata's Shared Nothing Advantages

One of the key advantages of Teradata is its underlying shared nothing architecture. Over the years this shared nothing architecture has evolved from being hardware-specific to being imposed by the software alone. In support of this shared nothing concept, per table data in the traditional Teradata system is spread evenly across the AMPs, with each AMP owning and operating on its share of the data.

Reading or writing data in traditional Teradata systems has the built-in advantages of a master index/cylinder index/data block indexing structure that is a permanent part of each individual AMP. These local structures are essential to an AMP's performance, independence, and self-sufficiency.

5.1.1 The Challenge

On the VantageCloud Lake platform, it is expected that many OFS tables will be accessed from object storage and read into the AMPs of Compute Cluster nodes. The AMPs on these nodes do not contain persistent block storage and there may be a differing number of AMPs participating in the read activity across different-sized clusters.

At first glance, the link between the AMP and the data in OFS appears to be broken, as illustrated in Figure 17. This could be seen as a potential deviation in the shared nothing story that Teradata has always adhered to through the years. Without AMP ownership of data, how is it possible for each AMP to be self-sufficient? And what happens to AMP-enabled parallelism when the data is uncoupled from the parallel units?

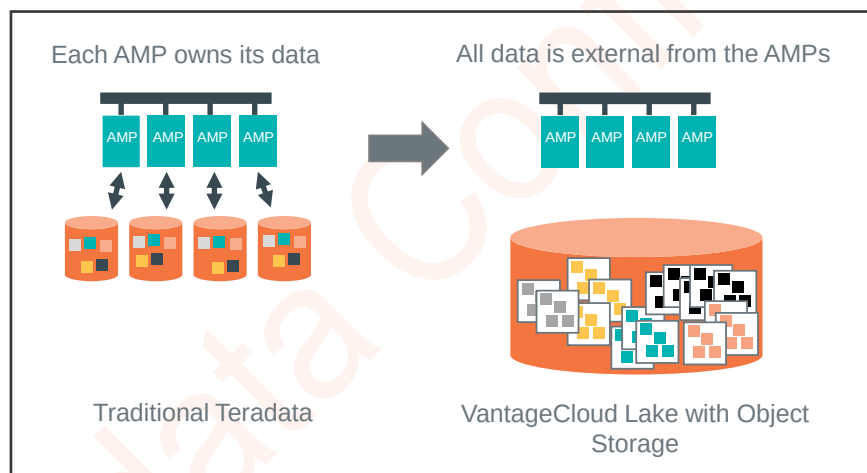


Figure 17: Uncoupling the Data Ownership from the AMPs.

5.1.2 The “Dynamic” Solution

OFS, which governs tables held in VantageCloud Lake object storage, has been designed in a manner to preserve and extend Teradata's shared nothing advantages. One way it does this is by dynamically assigning objects to AMPs at the time a query executes, providing a sort of “temporary ownership” of object data for the life of a query. This is not a random or a round-robin assignment. Instead, it is a well-planned, intelligent association that relies on existing database conventions.

Hash buckets are the means of establishing this temporary AMP ownership of data. Just as with traditional Teradata, a hashing algorithm accepts a value and produces a hash bucket. That hash bucket number will be used to probe the hash map to identify a single AMP that will own all the objects associated with that hash bucket.

In OFS, using hash buckets gives the VantageCloud Lake platform flexibility in how data is processed by the AMPs. Temporary AMP ownership of objects supporting NoPI tables is determined by hashing the value of the logical unique identifier of each object.

This dynamic association of objects to AMPs provides these three benefits:

1. The same row will end up on the same AMP for multiple accesses of a table when the reading cluster configuration remains the same.
2. Flexibility is established that supports configurations with differing numbers of AMPs reading the same table at different times.
3. NOS cache will perform more optimally if the same object is assigned to the same AMP.

Marrying objects to AMPs dynamically at the time a query is executing is in contrast to a traditional Teradata system, which, while it also relies on hash buckets, fixes its hash bucket-to-AMP assignments in a permanent way: That association is wired in at hardware configuration time.

6 Conclusion

VantageCloud Lake provides a different architecture compared to traditional Teradata platforms. Some of the key advantages of this new architecture include the following:

- Current capabilities of Teradata are carried forward, including the mature optimizer and high-performance analytics.
- Many existing Teradata features have been re-purposed so that they are used in new ways, for example QueryGrid conventions are enhanced to become the Query Fabric component that supports communication and passing of data and metadata between Primary and Compute Clusters.
- Elasticity is automatic and has been built into the platform. Expansion and contraction of processing power takes place without any down time, and can be customized for the different departments' needs..
- The increased reliance of object storage has led to a new Object File System that can manage and streamline access to customer data that has been moved to VantageCloud Lake's object storage.
- A new time-travel capability has been introduced, based on versioning tables that are stored under the control of OFS.
- All upgrades and enhancements are delivered as a service, eliminating manual steps for provisioning and services.

This orange book describes the initial release of the VantageCloud Lake Platform as it appears at this point in time. These capabilities should be considered as phase one. Additional enhancements and new features will be forthcoming as VantageCloud Lake evolves.